

ENSC 254 Lab Assignment 5

Important Logistics:

- Some general grading logistics and lab computer access instructions have been posted on our course website.
- Lab 5 weighs 9% of the final marks. It includes 180 points in total, which will be scaled to 9% of the final marks. Lab 5 is more open-ended and has a competition component.
- Lab 5 will be done and graded per 2-student group, as detailed in the lab logistics. Some points breakdown and guideline on workload division are provided later in the description.
 - Please make sure your code can compile and run correctly on the lab computers (i.e., FASRLA Linux computers). If your code cannot compile on the lab computers, then you get 0 mark for lab 5. If your code can compile, but cannot run correctly, then you only get the points where your code runs correctly on the lab computers.
- **Make sure you read through the entire spec and review the “Hardware-Aware Software Optimization” tutorial before starting lab 5.**

Introduction of Lab 5:

The purpose of this assignment is to enhance your understanding of hardware-aware software optimizations learned throughout ENSC 254 such that you can write well-optimized software on commodity CPUs. This includes three major optimizations: 1) data tiling (using loop tiling) to leverage CPU caches to achieve better memory system performance, 2) vectorization (using X86 SIMD intrinsics) to leverage CPU SIMD instructions to achieve better fine-grained data-level parallelism; note this is a single-core optimization, and 3) parallelization (using OpenMP multithreading) to leverage multiple CPU cores to achieve thread-level parallelism. Specifically, you will optimize the widely used General Matrix Multiply (GeMM) algorithm (backbone for deep learning) on our lab machine, which is a 14-core X86 CPU, by incrementally applying these three major optimizations. We will include some marks for the performance competition in lab 5.

Take the challenge and have fun :-)

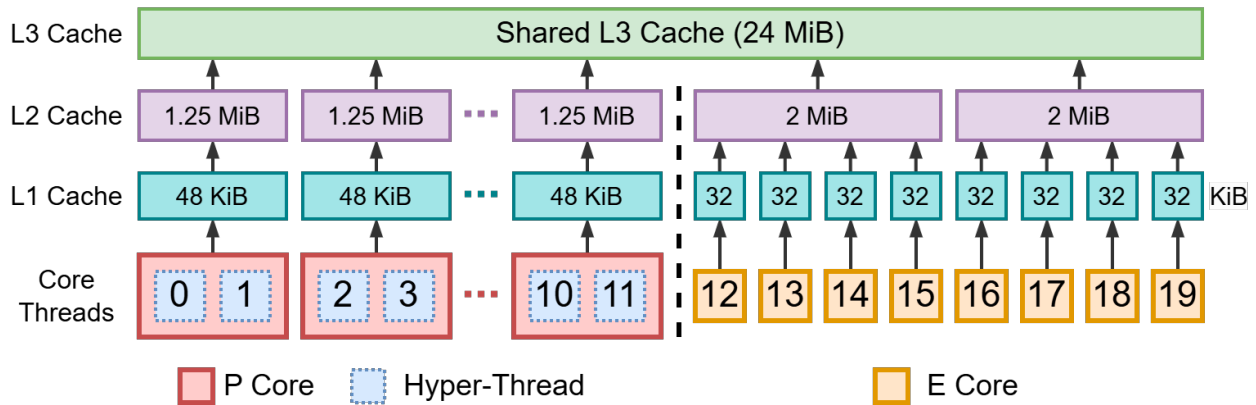
Hardware Configurations

In lab 5, you will optimize the software on our lab machine with the following CPU configurations.

# of CPU cores	6 performance cores (P-core, 2 hyper-threads per P-core) + 8 efficient cores (E-core). 20 hyper-threads in total.
SIMD support	It supports AVX2 (256-bit), but not AVX-512 (512-bit)

Our (FAS-RLA) lab machines use the Intel(R) Core(TM) i5-13500 CPU. The i5-13500 CPU has 6 performance cores (P-cores) which are hyper-threaded and 8 efficient cores (E-cores) which are not hyper-threaded, resulting in 20 hyper-threads in total. It supports AVX2 (256-bit) instructions, but not AVX-512 (512-bit) instructions. Such information can be retrieved using the “cat /proc/cpuinfo” command on a Linux machine. Moreover, the “lscpu --all --extended” command can figure out which logical cores (i.e., hyper-threads, see the “CPU” column of the output) belong

to which physical core (see the “CORE” column of the output), thus figure out which cores are P-cores (hyper-threaded) and which are cores are E-cores (not hyper-threaded). Specifically, for the i5-13500 CPU, CPU 0-11 (i.e., CORE 0-5) are P-cores, and CPU 12-19 (i.e., CORE 6-13) are Ecores, which is also shown in the figure below.



For the caches, our lab machine includes a three-level cache, as shown in the figure above. Each P-core has a 48 KiB private L1 data cache and a 1.25 MiB private L2 cache. Each E-core has a 32 KiB private L1 cache, and every 4 E-cores share a 2 MiB L2 cache. All the P-cores and E-cores share a 24 MiB L3 cache. The detailed configurations are shown in the table below and the figure above, which can be retrieved using the cache-info.sh script provided in lab 5, by running “./cacheinfo.sh <0-19>”.

Cache config	Cache for P-cores	Cache for E-cores
Level 1 cache size	48 KiB private, associativity = 12	32 KiB private, associativity = 8
Level 2 cache size	1.25 MiB private, associativity = 10	2 MiB shared, associativity = 16
Level 3 cache size	24 MiB shared cache shared by all cores, associativity = 12	
Cache line size	64 bytes per cache line for all three levels of cache	

GeMM Algorithm

The core computation of general matrix multiplication (GeMM) is $C = \alpha * A * B + \beta * C$, where A, B, and C are 4096*4096 size matrices, and alpha and beta are two constants. **DO NOT change the matrix size; otherwise, you automatically get a ZERO mark for lab 5.**

The core code is in the following function in mm.cpp file. We have different suffixes of the gemm function, which denote different versions of the function that you will implement.

```
void gemm_suffix(float C[NI*NJ], float A[NI*NK], float B[NK*NJ], float alpha, float beta) {
    int i, j, k;
    for (i = 0; i < NI; i++){
        for (j = 0; j < NJ; j++){
```

```

        C[i*NJ+j] *= beta;
    }
    for (j = 0; j < NJ; j++){
        for (k = 0; k < NK; ++k) {
            C[i*NJ+j] += alpha * A[i*NK+k] * B[k*NJ+j];
        }
    }
}
}
}

```

gemm_base is the baseline version of the GeMM algorithm, which gives you the correct sum of array C value (27789682688.000000 for the given 4096*4096 size matrices) and the baseline execution time (more than 10 minutes for the given 4096*4096 size matrices). Please do not change the gemm_base function.

Your TODO list:

In lab 5, you will implement the following functions in main.c. The function prototypes have been given; don't change them. Please only change the function bodies. If you want to add extra helper functions, it's ok; but don't change existing function prototypes.

- **gemm_tile:** Inside this function, apply the data tiling optimization to improve the cache locality and thus the memory access performance.
- **gemm_tile_simd:** Inside this function, on top of the data tiling optimizations, apply the vectorization (SIMD) optimizations by using the X86 intrinsics. Note our lab machine only supports up to the 256-bit AVX2 instructions, and does not support 512-bit AVX-512 instructions.
- **gemm_tile_simd_par:** Inside this function, on top of the data tiling and vectorization optimizations, apply the parallelization optimization using OpenMP pragmas to parallelize the execution of the loop(s).
- **gemm_perf:** Inside this function, implement any of the above, along with your own optimization techniques to achieve the best performance. You must not violate any of the rules described in the performance competition rubric of the grading section.
- Besides the coding, you will also need to write a simple document (.pdf format) to document the speedup achieved through each optimization (i.e., each function) compared to the gemm_base baseline. Include this simple document (in .pdf format) in your lab5 folder. See more details in grading.

Performance vs Efficient Cores:

As mentioned earlier, the Intel(R) Core(TM) i5-13500 CPU has a hybrid architecture with two types of cores. The 6 P-cores are designed for maximum performance and can run at a max frequency of 4.8 GHz, while the E-cores are optimized for power efficiency with a max frequency of 3.5 GHz. By default, OMP will use all available cores, however, it is possible, and sometimes

beneficial to limit the number of threads and assign them to specific cores. A `set_affinity(int core_id)` helper function is provided, which will assign the current thread to the specified core.

For example, if you wish to only use P cores (0-12, since each is hyperthreaded) you can use this code at the beginning of your parallel functions (**only for `gemm_tile_simd_par` and `gemm_perf`**):

```
void gemm_suffix(float C[NI*NJ], float A[NI*NK], float B[NK*NJ], float alpha, float beta) {  
    omp_set_num_threads(12); // set default number of threads for omp blocks  
    #pragma omp parallel  
    {  
        set_affinity(omp_get_thread_num());  
    }  
    // ... remaining code  
}
```

This code sets each thread from 0-12 to use core 0-12 respectively. You may modify this code to assign threads to cores as you wish. You may also opt to not specify thread cores at all, and leave the OMP default.

Basic Commands and Testing:

- To build your GeMM, type the following command on a lab computer (all commands are typed when you are inside the lab5 folder):

make

- Before measuring the execution time of your code, use the following command:

top

This command shows you the currently-running programs on the computer. Note that there are a large number of commands run by “root”: these are services required by the Linux, and other IT things. If there is another process that uses a significant amount of CPU, you will see a large number in the “%CPU” column. This means you should wait for this command to finish, or move to another computer.

- To measure the execution time of each version, type the following command:

./mm 0

This will run the `gemm_base` function, print out its execution time, and whether the result is correct (always correct for `gemm_base`).

Similarly, you can run

./mm 1

To run the `gemm_tile` function, print out its execution time, and whether the result is correct. Similarly, 2 is for `gemm_tile_simd`, 3 is for `gemm_tile_simd_par`, and 4 is for `gemm_perf`.

Grading:

- **[40 points] Data tiling optimization. If the final result is incorrect, you will get 0 point.**
 - [20 points] Your `gemm_tile` function indeed has the data tiling optimization and works correctly (i.e., giving correct result and better performance over baseline).
 - [20 points] Try at least five different tile sizes (note the tiling size along each dimension doesn't need to be the same, they can be different) and document their speedup compared to the baseline version using a bar graph with data labels. For your final submission, use the best tile size that gives you the best performance.
- **[25 points] Vectorization (SIMD) optimization. If the final result is incorrect, you will get 0 point.**
 - [20 points] Your `gemm_tile_simd` function indeed has both the data tiling and vectorization (SIMD) optimization, and works correctly (i.e., giving correct result and better performance over tiling version).
 - [5 points] Document its speedup compared to the baseline version.
- **[55 points] Parallelization optimization. If the final result is incorrect, you will get 0 point.**
 - [20 points] Your `gemm_tile_simd_par` function indeed has all three optimizations, and works correctly (i.e., giving correct result and better performance over vectorization/SIMD version).
 - [20 points] Try different number of threads (2, 4, 6, 8, 10, 12, ... all the way to 20). Document their speedups compared to the single-thread version (with both data tiling and vectorization optimizations) using a bar graph with data labels. For your final submission, use the best number of threads that gives you the best performance.
 - [10 points] After you choose the best number of threads, try to re-adjust the tile size again (at least five different tile sizes) and document their speedup compared to the original tile size version using a bar graph with data labels. For your final submission, use the best tile size (with the best number of threads) that gives you the best performance. Note tile sizes under single-thread and multi-thread programs may be different.
 - [5 points] Document the step-by-step speedup over the baseline version using a bar graph with data labels, including the (single-thread) data tiling version with the best tile size, the vectorization version, and the parallelization version with the best number of threads and tile size.
 - Note there should be three different bar graphs and they are comparing to different versions, as specified above.

- **[60 points] Performance competition.** The TA will run your `gemm_perf` function three times to get the average execution time on the same machine. The performance competition marks are given based on the following standard.
 - To be eligible for the performance competition, your final program must give the correct result; otherwise, you will get 0 point for the performance competition.
 - You CANNOT change the Makefile; otherwise, you will get 0 point for the performance competition.
 - Include other optimizations as well, such as loop unrolling to improve the instruction-level parallelism, to further push forward your program performance. Please document them in your report if you decide to use other optimizations. But keep in mind, you need to make sure your final result is correct; otherwise, you will get 0 point for the performance competition.
 - Performance competition points based on your average execution time t (measured by TA)
 - $t < 0.7$ seconds : 60 points
 - $0.7 \text{ seconds} \leq t < 0.8 \text{ seconds}$: 55 points
 - $0.8 \text{ seconds} \leq t < 0.9 \text{ seconds}$: 50 points
 - $0.9 \text{ seconds} \leq t < 1.0 \text{ seconds}$: 45 points
 - $1.0 \text{ seconds} \leq t < 1.1 \text{ seconds}$: 40 points
 - $1.1 \text{ seconds} \leq t < 1.2 \text{ seconds}$: 35 points
 - $1.2 \text{ seconds} \leq t < 1.4 \text{ seconds}$: 30 points
 - $1.4 \text{ seconds} \leq t < 1.6 \text{ seconds}$: 25 points
 - $1.6 \text{ seconds} \leq t < 1.8 \text{ seconds}$: 20 points
 - $1.8 \text{ seconds} \leq t < 2.0 \text{ seconds}$: 15 points
 - $2.0 \text{ seconds} \leq t < 2.2 \text{ seconds}$: 10 points
 - $2.2 \text{ seconds} \leq t < 2.4 \text{ seconds}$: 5 points
 - $t \geq 2.4 \text{ seconds}$: 0 points

Assignment Submission:

Your lab assignment 5 will be submitted electronically. You will need to submit a single ***YOUR_LAB_GROUP.zip*** file (e.g., LA01_01.zip). This format makes it easy for TA to do auto-grading. Please double check the format correctness. **If you fail to comply with this format, you will lose 10% of the total points of this lab.** To zip your files in Linux,

1. *Go to your lab5 folder*
2. *make clean //make sure you clean up your files*
3. *cd .. //go one level up*
4. *mv lab5 YOUR_LAB_GROUP //rename your lab5 folder using your lab group (e.g., LA01_01)*

5. *zip -r YOUR_LAB_GROUP.zip YOUR_LAB_GROUP //zip all your lab5 files into a single .zip*

Submission Deadline:

You need to meet the deadline: every 10 minutes late for submission, you lose 10% of the points; that is, 100 minutes late, you will get zero for this lab.

Lab Demonstration:

To finalize your lab 5 points, you will have to demo your lab code to your TA in the lab sessions that you. Due to the large enrollment, TA will **randomly** select some students (about 1/2 student groups) to do the lab 5 demo. If you are NOT selected to do the demo, the auto-graded points are your final points.

For those who are selected to do the demo, each student group has around 10 minutes to explain their code to the TA. Only code from your submission is allowed in the lab demo. If you fail to do the demo (without a medical note), you will be awarded zero on this lab assignment. **If it is determined that you do not understand your code being evaluated (the part you are responsible for based on your workload distribution), you will be awarded zero and considered as CHEATING on this lab assignment.** Also please show up in the demo day on time (TA will send out your scheduled time), otherwise you will lose 10% of the lab 5 points.